TEC | Tecnológico de Costa Rica

Tarea #1

Gramática BNF

Estudiante

Geovanni González Aguilar

Escuela de Ingeniería en Computación

Curso

IC5701 - Compiladores e Intérpretes

Profesor

Allan Rodríguez Davila

Grupo

60

II Semestre 2026

Descripción del problema

En la actualidad, la industria de los semiconductores y los sistemas empotrados experimenta un crecimiento constante, lo que genera una demanda de microchips cada vez más complejos. Esta evolución tecnológica requiere de herramientas de software especializadas que permitan una configuración eficiente, ligera y potente de estos dispositivos. Los lenguajes de programación de propósito general suelen incluir sobrecargas innecesarias para entornos de recursos limitados, lo que dificulta la optimización en el nivel de hardware.

Ante esta situación, un grupo de desarrolladores se ha propuesto la creación de un nuevo lenguaje de programación de paradigma imperativo, diseñado específicamente para la configuración de chips y sistemas empotrados. Este lenguaje debe ser:

1. **Ligero:** Para operar en dispositivos con memoria y procesamiento restringido.
2. **De tipado fuerte y explícito:** Para garantizar la seguridad y previsibilidad del código en tareas críticas de hardware.
3. **Expresivo:** Capaz de manejar operaciones aritméticas complejas (incluyendo notación exponencial y fraccionaria) y estructuras de datos bidimensionales.

El problema central de este proyecto radica en la etapa inicial de diseño: la definición de una **Gramática en Notación de Backus-Naur (BNF)**. Esta gramática debe formalizar los aspectos léxicos y sintácticos del lenguaje, permitiendo la creación de funciones, estructuras de control manejo de diversos tipos de variables y una jerarquía clara de expresiones matemáticas y lógicas.

Diseño del programa

Símbolo Inicial

- <programa>

Lista de Terminales (Tokens)

Los terminales son los símbolos básicos que el lenguaje reconoce y no se pueden dividir más:

- **Palabras Reservadas:** int, float, bool, string, char, arr_int, arr_float, if, else, do, while, switch, case, default, break, return, cin, cout, empty.
- **Operadores Aritméticos:** +, -, *, /, %, ^, e, ++, --.
- **Operadores Lógicos:** @ (AND), # (OR), $ (NOT).
- **Símbolos de Control:** |: (Inicio Bloque), :| (Fin Bloque), << (Inicio Índice), >> (Fin Índice), <| (Inicio Paréntesis/Func), |> (Fin Paréntesis/Func).

- **Delimitadores:** ~ (Separador), <- (Asignación), ! (Fin de sentencia), , (Coma), : (Dos puntos).
- **Literales:** 0-9 (Dígitos), a-z / A-Z (Letras), ' (Comilla simple), " (Comilla doble), true, false, 1, 0.
- **Comentarios:** ii (Línea), {- y -} (Multi-línea).

Lista de No Terminales

Son las reglas abstractas que definen la estructura lógica:

- <programa>, <lista_global>, <procedimiento_main>, <bloque>, <lista_sentencias>, <sentencia>, <sentencia_simple>, <estructura_control>, <expresion>, <exp_logica>, <exp_relacional>, <exp_aritmetica>, <term_arit>, <fact_arit>, <valor>, <tipo>, <nombre_variable>, <acceso_indice>, <arreglo_bidimensional>, entre otros.

Producciones

El programa se rige por la siguiente jerarquía de derivación:

1. **Estructura Global:** Un programa consiste en una lista opcional de funciones globales seguida obligatoriamente por el procedimiento empty ~ __main__ <| |>.
2. **Manejo de Bloques:** Todo conjunto de sentencias en funciones o estructuras de control debe estar delimitado por los símbolos de chip |: y :|.
3. **Sistema de Expresiones (Sin ambigüedad):**
    - La gramática utiliza una estructura de niveles para respetar la **precedencia matemática usual**:
        - Nivel 1: Expresiones Lógicas (@, #).
        - Nivel 2: Expresiones Relacionales (Funciones equal, less_t, etc.).
        - Nivel 3: Expresiones Aritméticas Aditivas (+, -).
        - Nivel 4: Expresiones Aritméticas Multiplicativas (*, /, %).
        - Nivel 5: Operadores Unarios y Potencia (++, --, -, ^).
        - Nivel 6: Primarios (Paréntesis <| |>, variables, arreglos y literales).
4. **Tipado Fuerte:** Las reglas de creación exigen el formato <tipo> ~ <id>, asegurando que cada variable tenga un tipo definido desde su nacimiento.
5. **Notaciones Especiales:** * Se incluyen reglas específicas para derivar números con mantisa y exponente (100e10).
    - Se incluye una regla de fraccionario que permite la división literal de dos enteros como un valor constante.

Análisis de resultados

Lecciones Aprendidas

- **Manejo de la Recursividad:** Se comprendió la importancia de evitar la recursividad por la izquierda en gramáticas destinadas a parsers descendentes (como los que suelen emular los validadores BNF), transformando las reglas para usar recursividad por la derecha sin alterar la lógica del lenguaje.

- **Jerarquía de Expresiones:** Se aprendió a estructurar los niveles de precedencia (Aritmética $\rightarrow$ Relacional $\rightarrow$ Lógica) de forma que el árbol de derivación respete las reglas matemáticas usuales, evitando las estructuras "prohibidas" mencionadas en el enunciado.

- **Sintaxis Contextual para Hardware:** Al diseñar un lenguaje para chips, se asimiló cómo el uso de delimitadores explícitos (como |: y :|) y separadores obligatorios (~) facilita la fase de análisis léxico, reduciendo la ambigüedad en comparación con lenguajes de propósito general.

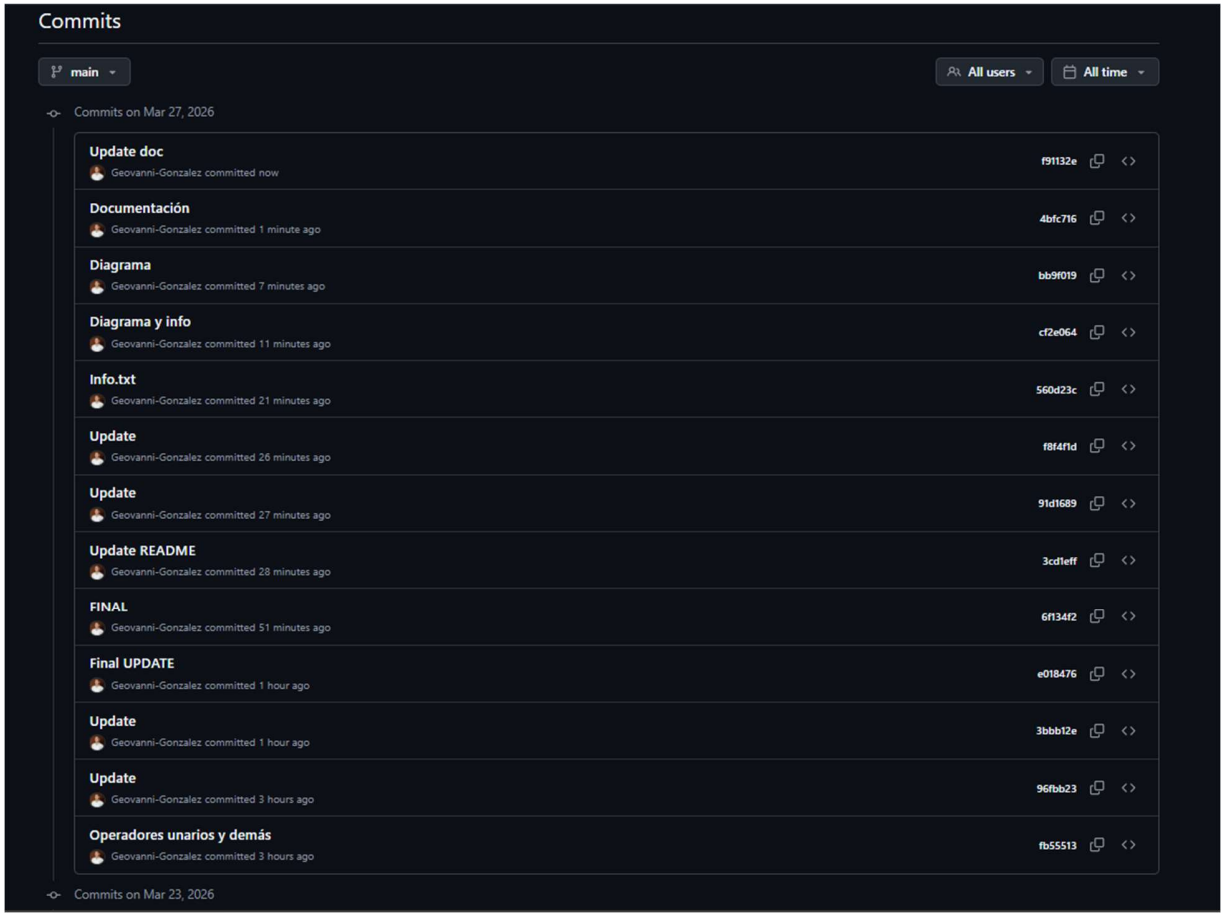
- **Uso de Herramientas de Validación:** La interacción con el BNF Playground permitió identificar errores sutiles en la definición de terminales opcionales y la necesidad de manejar símbolos vacíos (épsilon) de forma explícita según el motor de parsing utilizado.

Objetivos alcanzados

- **Definición Léxica Completa:** Se implementaron todos los tipos de datos requeridos, incluyendo casos especiales como la notación exponencial positiva y los tipos fraccionarios.
- **Estructuras de Control:** Se logró integrar satisfactoriamente las estructuras if-else, do-while y switch, respetando el uso del separador ~ y las condiciones booleanas.
- **Operadores como Funciones:** Se cumplió con el requerimiento **(i)** de definir los operadores relacionales con sintaxis de función (ej. equal<|op1,op2|>), integrándolos correctamente dentro de la cadena de producción de expresiones.
- **Punto de Entrada Único:** Se definió la producción obligatoria empty ~ __main__ <| |>, asegurando que todo programa válido tenga una estructura de inicio estandarizada.
- **Manejo de Arreglos:** Se permite la creación, modificación y acceso de arreglos bidimensionales con la simbología << >> y corchetes anidados.

Bitácora

Historial de commits (Github)



Link a Github

https://github.com/Geovanni-Gonzalez/GramaticaBNF

Herramientas

https://bnfplayground.pauliankline.com/

https://www.bottlecaps.de/rr/ui